Zuu Crew
Machine Learning
Acadamy

ZERO TO DATA ENGINEER SPECIALIZATION

Mini Project 02

ABC Hub – Evolving the Data Platform

Continuation of Mini Project 01 | Data Engineering Bootcamp | Zuu Crew Machine Learning Academy

Project Context

ABC Hub has successfully implemented its initial analytical data platform. The company is now experiencing increased data volume and requires a more reliable, maintainable, and production-ready data platform. The Data Engineering team is evaluating Apache Airflow for workflow orchestration while continuing to use the existing solution as the foundation.

Your task is to enhance the solution you built in Mini Project 01. You should not create an unrelated new project. Your existing ABC Hub business scenario, data model, analytical requirements, and database design should be reused and improved.

Project Objectives

- Understand the role of Apache Airflow in a modern data platform.
- Make an evidence-based technology decision between Apache NiFi and Apache Airflow.
- Apply Medallion Architecture to the existing ABC Hub platform.
- Implement and demonstrate Slowly Changing Dimensions (SCD).
- Build an Airflow DAG using appropriate operators, dependencies, and Jinja templating.
- Design incremental and idempotent data processing.
- Apply data quality, retry, backfill, audit, and recovery concepts.

Important Implementation Guidance

- Use your Mini Project 01 solution as the starting point.
- Do not replace the entire solution simply to complete this project.
- Use Airflow's built-in operators and SQL wherever practical.
- Do not write custom Python code for transformations or orchestration unless there is a genuine technical reason.
- Use Airflow Jinja templating to make execution dates, processing windows, and other runtime values dynamic.
- Your solution must demonstrate idempotency: re-running the same data should not create duplicate analytical records or unnecessary SCD versions.

Question 1 – Airflow Fundamentals and Technology Decision

20 Marks

ABC Hub currently uses Apache NiFi for the ETL pipeline developed in Mini Project 01. As the platform grows, the Data Engineering team is evaluating Apache Airflow for workflow orchestration. Before introducing Airflow, the team wants to understand what problem it solves and whether it should replace NiFi or work alongside it.

Part A – Airflow Fundamentals

Explain the following Apache Airflow concepts and describe how they work together when executing a data pipeline:

- DAG
- DAG Run
- Task
- Task Instance
- Scheduler
- Executor
- Operator
- Sensor
- Connection
- XCom
- Task dependency
- Scheduling
- Retry and failure handling

Part B – NiFi vs Airflow Decision Analysis

ABC Hub does not want to select a technology simply because it is popular. Evaluate Apache NiFi and Apache Airflow against the following requirements and make a justified technology decision.

Decision Factor	Importance	Apache NiFi	Apache Airflow	Preferred	Justification
Data ingestion	High				
Data movement	High				
Workflow orchestration	High				
Complex task dependencies	High				
Scheduling	High				
Retry management	High				
Failure handling	High				
Incremental processing	High				
Backfilling	High				
SQL-based workflows	High				
Jinja templating	Medium				
Parameterisation	Medium				

Decision Factor	Importance	Apache NiFi	Apache Airflow	Preferred	Justification
Pipeline monitoring	High				
Data transformation	High				
Streaming / event processing	Medium				
File-based processing	Medium				
Maintainability	High				
Reusability	High				
Scalability	High				
Dependency management	High				

Scoring: Use a consistent scoring scale such as 1 = Poor, 2 = Limited, 3 = Acceptable, 4 = Good, and 5 = Excellent. Apply appropriate importance weighting and calculate the final score for each technology.

Part C – Architectural Decision

- Compare a NiFi-only architecture, an Airflow-only architecture, and a NiFi + Airflow architecture.
- Based on your decision table, recommend the most appropriate architecture for ABC Hub.
- Clearly explain why the selected technology should perform ingestion, transformation, orchestration, or other responsibilities.
- Identify the main strengths and limitations of your selected approach.

Question 2 – Medallion Architecture and Historical Dimensional Modelling

25 Marks

ABC Hub wants to improve the organisation of its data platform and introduce a Medallion Architecture. At the same time, management requires the analytical platform to preserve historical changes to important business dimensions.

Part A – Medallion Architecture

- Redesign your Mini Project 01 architecture using Bronze, Silver, and Gold layers.
- Explain the purpose of each layer.
- Identify what data should be stored in each layer.
- Identify where cleansing, validation, standardisation, and business transformations should occur.
- Identify where the Fact and Dimension tables from Mini Project 01 should exist.
- Explain how Medallion Architecture improves the existing platform.

Part B – Slowly Changing Dimensions (SCD)

ABC Hub now needs to preserve historical changes. For example, a customer may initially live in Colombo and later move to Kandy. Management must be able to analyse both the customer's current information and the information that was valid when a historical transaction occurred.

- Identify which dimensions in your existing analytical model require historical tracking.
- Select the appropriate SCD strategy for each affected dimension.
- Justify the selected SCD strategy.
- Redesign the affected dimensions using appropriate surrogate keys and historical attributes.
- Demonstrate the historical change using sample data.
- Explain how the previous version of a dimension record is preserved.
- Explain how Fact records continue to reference the correct historical dimension version.
- Identify any conformed dimensions shared across multiple Fact tables.
- Update your Star Schema diagrams to reflect the changes.

Your design should support questions such as:

- What is the customer's current information?
- What was the customer's information when the transaction occurred?

Question 3 – Building an Airflow DAG with Incremental and Idempotent Processing

30 Marks

ABC Hub now wants Apache Airflow to orchestrate the enhanced data platform. The DAG must process the Medallion layers, perform the required historical dimension processing, and safely support repeated execution.

Part A – DAG Design

Design an Airflow DAG that represents the following logical workflow:



- Implement appropriate task dependencies.
- Use suitable Airflow operators for the work being performed.
- Explain why each task exists and why the dependency order is required.

Part B – Jinja Templating

The DAG must support different execution dates and processing windows without manually changing the DAG code.

- Use Airflow Jinja templating for execution date, processing date, data interval, source extraction window, and other appropriate runtime values.
- Demonstrate how a templated value is passed into a SQL query or task.
- Explain why templating is preferable to hardcoding dates and runtime values.

Part C – Avoid Unnecessary Python

The implementation should not use Python code for transformations or orchestration when the requirement can reasonably be handled using an existing Airflow operator, SQL, or Jinja template.

- Identify which parts of your pipeline can be implemented using built-in Airflow operators.
- Use SQL for database transformations where appropriate.

- Use Jinja templates for dynamic runtime values.
- Use custom Python only when you can clearly justify why an existing operator or SQL approach is insufficient.

Part D – Incremental Processing and Idempotency

- Explain what an incremental data pipeline is and how your DAG determines which data needs to be processed.
- Define the mechanism used to identify already processed data, such as a processing date, watermark, source timestamp, or batch identifier.
- Explain what makes a data pipeline idempotent.
- Implement loading logic that prevents duplicate analytical records.
- Ensure that re-running the same data does not create unnecessary SCD versions.
- Demonstrate the behaviour by running the same data more than once.

> ***Required demonstration:*** *Run 1: 100 records processed → Run 2: same data processed again → Expected final result: 100 valid analytical records, not 200.*

Question 4 – Productionising the Airflow Data Platform

25 Marks

ABC Hub is preparing to operate the analytical platform as a production data pipeline. The Data Engineering team requires reliability, data quality, recovery, auditability, and maintainability.

Part A – DAG Reliability

Configure and explain appropriate settings for:

- DAG scheduling
- Start date and execution behaviour
- Retries
- Retry delay
- Task dependencies
- Task timeouts
- Failure handling
- Success/failure notifications where applicable

Part B – Backfilling Historical Data

ABC Hub discovers that data for several previous processing dates was not loaded because of an operational failure.

- Demonstrate how the Airflow DAG can process the missing historical dates.
- Use execution context and Jinja templating so that the same DAG logic can process different dates.
- Explain why backfilling is useful in a production data platform.
- Demonstrate that backfilling does not create duplicate analytical records.

Part C – Data Quality

Introduce validation tasks into the DAG. Your solution should include appropriate checks such as:

- Null and mandatory-field checks
- Duplicate detection
- Referential integrity
- Valid dates
- Valid monetary values
- Valid business statuses
- Fact and Dimension consistency

> ***Requirement:*** *Define what should happen when a data quality check fails. The pipeline should not silently load invalid data.*

Part D – ETL Audit

Design and implement an audit mechanism that records appropriate information for each DAG run, including:

Field	Purpose
batch_id	Identify the processing batch
dag_id	Identify the Airflow DAG
run_id	Identify the specific DAG run
start_time	Record processing start
end_time	Record processing completion
status	Record success/failure
source_record_count	Records extracted
processed_record_count	Records processed
inserted_record_count	Records inserted
updated_record_count	Records updated
rejected_record_count	Records rejected

Part E – Final Architecture

Produce the final ABC Hub architecture incorporating everything developed in Mini Projects 01 and 02.

- Operational systems / source database
- Data ingestion
- Bronze layer
- Apache Airflow orchestration
- Silver layer
- Data quality checks
- SCD processing
- Gold layer
- Fact and Dimension tables
- Data marts / analytical consumption
- BI or business users
- Error handling and audit mechanisms

Final Technology Decision

Update your technology decision table from Question 1 based on what you learned while implementing the solution. State whether your final architecture should use NiFi, Airflow, or both, and explain the responsibility of each technology.

Submission Requirements

- A report containing answers to Questions 1–4.
- Updated architecture diagrams.
- Updated dimensional model / Star Schema diagrams.
- SCD design and implementation evidence.
- Apache Airflow DAG files.
- Jinja-templated SQL or task configurations used by the DAG.
- PostgreSQL SQL scripts required to create or update the analytical structures.
- Evidence of incremental processing and idempotent execution.
- Evidence of data quality validation and rejected-record handling.
- Evidence of retries, backfilling, and audit information.
- Screenshots showing successful Airflow DAG execution and relevant PostgreSQL results.
- A README describing setup, dependencies, configuration, execution instructions, assumptions, and technology decisions.

Implementation Principles

- **Build on Mini Project 01:** reuse and improve your existing ABC Hub solution.
- **Decision before implementation:** justify technology choices using the decision matrix.
- **Prefer declarative solutions:** use SQL and Airflow operators where practical.
- **Use Jinja templating:** avoid hardcoded execution dates and processing windows.
- **Design for retries:** the same task should be safe to execute again.
- **Design for history:** preserve required dimension history correctly.
- **Do not hide failures:** invalid data and failed tasks must be visible and traceable.
- **Production mindset:** design for repeatability, recovery, monitoring, and maintainability.

> ***Final Objective:*** *Transform the ABC Hub solution from a basic analytical ETL implementation into a reliable, orchestrated, historical, and production-oriented data platform.*